

Bitácora de Desarrollo — Proyecto PokéTienda Pearl

Asignatura: DSY1104 — Desarrollo Web Frontend
Proyecto: Tienda online temática Pokémon — Edición Perla
Versión del documento: 1.0
Fecha: 2025
Responsable técnico: Desarrollador Fullstack contratado
Cliente: Equipo de desarrollo DSY1104 (Gabriel Flores, Daniel Rangel, Hernaldo Silva)

1. Objetivo de la bitácora

Dejar constancia formal y trazable de las actividades realizadas sobre la capa de estructura (**HTML5**) del proyecto PokéTienda Pearl, así como de los hallazgos técnicos relevantes y de la hoja de ruta pendiente para la capa de presentación (**CSS3**) y comportamiento (**JavaScript**).

Esta bitácora sirve como:

- Evidencia de avance para la Evaluación Parcial 1 (IE1.1.1, IE1.1.2, IE1.1.3).
- Documento de traspaso entre integrantes del equipo.
- Base para la planificación de las siguientes iteraciones.

2. Contexto y antecedentes

El proyecto se desarrolla bajo los lineamientos establecidos en:

Documento	Rol
Especificación de Requisitos del Software (ERS) v1.0	Define requerimientos funcionales RF-01 a RF-13, reglas de validación y stack técnico
Propuesta de Identidad Visual "Pokémon Pearl" v2.0	Define paleta, tipografía, logo y guía de aplicación de componentes UI
DSY1104 Evaluación Parcial 1	Define la rúbrica de evaluación y los indicadores de logro IE1.1.1, IE1.1.2, IE1.1.3, IE1.2.1, IE1.2.2, IE1.3.1, IE1.3.2
Mockups Figura 2 a Figura 15	Referencia visual para cada vista (pública y administrativa)

Todas las decisiones de estructura tomadas respetan los tres documentos anteriores.

3. Actividades realizadas

3.1 Entregables HTML completados

Se desarrollaron **16 archivos HTML** organizados en dos módulos: **vista pública** (tienda) y **vista administrativa**.

3.1.1 Vista pública (raíz del proyecto)

#	Archivo	Requerimiento cubierto	Observaciones
1	<code>index.html</code>	RF-01 Página Principal	Header con nav + carrito, hero, productos destacados, footer
2	<code>productos.html</code>	RF-02 Productos	Grilla, toolbar (buscador + filtro + orden), 8 productos de respaldo
3	<code>detalle-producto.html</code>	RF-03 Detalle del Producto	Galería, selector cantidad, botón añadir, relacionados, breadcrumb de 3 niveles
4	<code>nosotros.html</code>	RF-07 Nosotros	Historia, misión/visión/valores, equipo (Gabriel, Daniel, Hernaldo), CTA
5	<code>blogs.html</code>	RF-08 Blogs	Listado tipo "Caso curioso" con botón "Ver caso"
6	<code>detalle-blog-1.html</code>	RF-09 Detalle de Blog	Artículo "El misterio de las perlas en la mitología de Sinnoh"
7	<code>detalle-blog-2.html</code>	RF-09 Detalle de Blog	Artículo "Cómo la Generación IV marcó un antes y un después en Pokémon"
8	<code>contacto.html</code>	RF-10 Contacto	Formulario (nombre, correo, comentario), info lateral, FAQ
9	<code>registro.html</code>	RF-05 Registro de Usuario	9 campos + selects dependientes región/comuna + términos
10	<code>login.html</code>	RF-06 Inicio de Sesión	Logo + nombre empresa encima del formulario, 2 campos

3.1.2 Vista administrativa (carpeta `admin/`)

#	Archivo	Requerimiento cubierto	Observaciones
1 1	<code>admin/home.html</code>	RF-11 Home Administrador	Sidebar vertical, bienvenida, métricas, accesos rápidos, alerta stock
1 2	<code>admin/productos.html</code>	RF-12 Mantenedor de Productos	Tabla completa, toolbar, paginación, botón "Nuevo producto"
1 3	<code>admin/producto-form.html</code>	RF-12 Crear/Editar Producto	Formulario con 8 campos, modo crear/editar vía <code>?id=</code>
1 4	<code>admin/usuarios.html</code>	RF-13 Mantenedor de Usuarios	Tabla con RUN, nombre, correo, región, rol; toolbar y paginación
1 5	<code>admin/usuario-form.html</code>	RF-13 Crear/Editar Usuario	Formulario con 9 campos, selects dependientes, modo crear/editar vía <code>?run=</code>

(Se numeran 15 archivos HTML; el decimosexto entregable, el archivo `variables.css` referenciado en la identidad visual, corresponde a la siguiente fase.)

3.2 Criterios técnicos aplicados

3.2.1 Semántica HTML5 (IE1.1.1)

Todas las vistas cumplen con la estructura semántica exigida por la rúbrica:

- `<header>` para el encabezado del sitio o de sección.
- `<nav>` para navegación principal y secundaria, siempre con `aria-label` descriptivo.
- `<main>` único por documento, con `id="contenido-principal"` para el skip-link.
- `<section>` para agrupaciones temáticas de contenido.
- `<article>` para contenido autocontenido: tarjetas de producto, entradas de blog, detalle de producto, formularios admin.
- `<aside>` para contenido complementario: sidebar admin, bloque de autor en blogs, info de contacto.
- `<footer>` en todas las vistas, con bloques de navegación, contacto y redes.
- Etiquetas auxiliares: `<figure>`, `<figcaption>`, `<address>`, `<time>`, `<code>`, `<blockquote>`, `<cite>`, `<details>`, `<summary>`.

3.2.2 Accesibilidad

- **Skip-link** en todas las páginas públicas y administrativas.

- **aria-current="page"** en el enlace activo de cada menú.
- **aria-labelledby** en cada sección para asociar títulos.
- **aria-describedby** en cada campo de formulario, apuntando a su mensaje de ayuda y a su mensaje de error.
- **aria-required="true"** en todos los campos obligatorios.
- **role="alert"** en los párrafos de error (anuncio inmediato por lectores de pantalla).
- **aria-live="polite"** en contadores y feedback general.
- **scope="col"** en cada `<th>` de tablas administrativas.
- **<caption class="visually-hidden">** en cada tabla para lectores de pantalla.
- **role="region" + tabindex="0"** en la envoltura de tablas, para habilitar scroll horizontal con teclado.

3.2.3 Separación de responsabilidades

- Estructura (HTML): 15 archivos `.html`.
- Presentación (CSS): **pendiente**, ver sección 5.
- Comportamiento (JS): **pendiente**, referenciado mediante `<script defer>`.

3.2.4 Coherencia visual entre páginas

- Header idéntico en las 10 vistas públicas: logo SVG con degradado perla, nav de 6 ítems, carrito con badge.
- Footer idéntico en las 10 vistas públicas: 4 columnas (marca, navegación, ayuda, contacto) + barra inferior.
- Sidebar admin idéntico en las 5 vistas administrativas: logo, navegación agrupada (Principal / Mantenedores / Sistema), botón de cierre de sesión.
- Topbar admin idéntico: botón hamburguesa, título dinámico, saludo + avatar del usuario.

4. Hallazgo técnico relevante

4.1 Síntoma

Durante la revisión de las vistas `admin/producto-form.html` y `admin/usuario-form.html`, el cliente reportó que se visualizaban como **páginas en blanco** al abrirlas directamente desde el sistema de archivos.

4.2 Causa raíz confirmada

Tras el análisis, se confirmó que **el proyecto aún no cuenta con archivos `.css` implementados** (`assets/css/variables.css`, `assets/css/styles.css`, `assets/css/admin.css`). Las vistas `admin/home.html`, `admin/productos.html` y

`admin/usuarios.html` mostraban contenido porque su estructura visible depende únicamente del flujo natural del HTML (tablas, sidebars con texto).

En cambio, `producto-form.html` y `usuario-form.html` concentran su contenido dentro de un contenedor `.formulario--admin`, cuyo layout en grilla depende íntegramente de las reglas CSS no existentes. Sin esas reglas:

- El `<form>` no tiene ancho definido.
- La grilla `formulario__grid` no se renderiza.
- El contenido queda colapsado a anchura cero o fuera del viewport.

Esto **no constituye un error del HTML** sino una consecuencia esperada de la ausencia de la capa de presentación.

4.3 Correcciones paralelas aplicadas

Aprovechando la revisión, se aplicaron tres mejoras al HTML detectadas en la auditoría:

1. **Añadido `<footer class="admin-footer">`** en las 5 vistas administrativas (anteriormente ausente; la rúbrica IE1.1.1 exige footer).
2. **Envuelto cada formulario admin en `<article>`** con `aria-labelledby` apuntando al título del formulario (contenido autocontenido y semánticamente correcto).
3. **Eliminado `<img src="">` vacío** en `producto-form.html`, reemplazado por `<div id="imagen-preview" hidden>` que JavaScript poblará dinámicamente al cargar una imagen.

5. Sigüientes pasos: capa CSS

5.1 Archivos a crear

Se deben crear los siguientes archivos en `assets/css/`:

Archivo	Propósito
<code>variables.css</code>	Tokens de diseño: colores, tipografía, espaciado, radios, sombras, gradientes
<code>styles.css</code>	Estilos globales de la vista pública (header, footer, botones, cards, formularios, hero, breadcrumb, blog, etc.)
<code>admin.css</code>	Estilos específicos del panel administrativo (layout grid, sidebar, topbar, tablas, formularios admin, badges)

5.2 Orden de importación

En cada HTML el orden ya está definido y debe respetarse:

```
<link rel="stylesheet" href="assets/css/variables.css">
<link rel="stylesheet" href="assets/css/styles.css">
<!-- Solo en admin: -->
<link rel="stylesheet" href="assets/css/admin.css">
```

La cascada garantiza que `admin.css` sobrescriba reglas de `styles.css` cuando sea necesario.

5.3 Contenido mínimo esperado

`variables.css`

Debe declarar los tokens definidos en la identidad visual v2.0:

- Colores: `--color-primario`, `--color-secundario`, `--color-acento`, `--color-acento-2`, `--color-fondo`, `--color-fondo-alt`, `--color-texto`, `--color-texto-sec`, `--color-exito`, `--color-error`, `--color-advertencia`, `--color-info`.
- Tipografía: `--font-titulos`, `--font-cuerpo`, `--font-acentos`.
- Espaciado: `--espacio-xs` a `--espacio-xl`.
- Bordes: `--radio-sm`, `--radio-md`, `--radio-lg`.
- Sombras: `--sombra-sm`, `--sombra-md`, `--sombra-lg`.
- Gradientes: `--gradiente-header`, `--gradiente-perla`.

`styles.css`

Debe cubrir, como mínimo:

- Reset y base (`box-sizing`, tipografía base, body con fondo oscuro).
- Header sticky con degradado `#4A148C` → `#1A052A`.
- Navegación principal con estado activo y hover amarillo `#FFC107`.
- Carrito con badge y contador.
- Hero con degradado cósmico y layout de 2 columnas.
- Grilla de productos con `card-producto`.
- Botones: `.boton`, `.boton--primario`, `.boton--secundario`, `.boton--bloque`.
- Formularios: `.campo`, `.campo__label`, `.campo__input`, `.campo__ayuda`, `.campo__error`, estados `:focus`, `.campo--error`, `.campo--exito`.
- Breadcrumb, badges, avisos, tabla pública si aplica.
- Footer con 4 columnas y barra inferior.

- Media queries para responsividad (breakpoints sugeridos: 768 px, 1024 px).

admin.css

Debe cubrir, como mínimo:

- Layout grid `.admin-layout` con sidebar fijo y contenido fluido.
- `.admin-sidebar` con fondo `#1A052A`, hover `#D63384`, activo con borde `#FFC107`.
- `.admin-topbar` sticky.
- `.admin-contenido` con fondo `#2A1B3D`.
- Tablas `.tabla--admin` con header `#4A148C` y filas alternas.
- Badges de estado (éxito, advertencia, error, info, primario).
- Formularios admin con grilla de 2 columnas (`.formulario__grid`).
- Toolbar, paginación, avisos.
- **Regla crítica para resolver el síntoma reportado:**

```
.formulario--admin,
.formulario__grid {
  display: block; /* o grid según diseño */
  opacity: 1;
  visibility: visible;
  height: auto;
  overflow: visible;
}

.formulario__grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: var(--espacio-lg, 1.5rem);
}
```

- Media queries específicas para el panel admin (sidebar colapsable en móvil).

5.4 Criterios de aceptación CSS

Al finalizar la capa CSS se debe verificar:

1. **Contraste AA** (4.5:1 mínimo) en todos los pares texto/fondo, conforme a la sección 3.3 de la identidad visual.
2. **Diseño responsivo** funcional en 3 breakpoints como mínimo (móvil, tablet, escritorio).
3. **Consistencia visual** entre las 10 vistas públicas y las 5 administrativas.
4. **Estado :focus visible** en todos los elementos interactivos (accesibilidad por teclado).
5. **Sin reglas display: none** sobre contenedores de contenido principal.

6. Sigüientes pasos: capa JavaScript

Aunque el alcance inmediato es CSS, se documentan los scripts a implementar en la fase siguiente para mantener la trazabilidad del proyecto:

Archivo JS	Propósito	Referenciado en
<code>productos.js</code>	Arreglo de productos y renderizado dinámico de <code>#lista-productos</code> y <code>#lista-relacionados</code>	productos, index, detalle-producto
<code>carrito.js</code>	Gestión de carrito en LocalStorage, badge del header, feedback	todas las vistas públicas
<code>main.js</code>	Toggle del menú móvil, comportamiento del header sticky	todas las vistas públicas
<code>contacto.js</code>	Validaciones del formulario de contacto	contacto
<code>registro.js</code>	Validaciones del formulario de registro	registro
<code>login.js</code>	Validaciones del formulario de login	login
<code>regiones-comunas.js</code>	Arreglo de regiones y comunas de Chile, populate de selects dependientes	registro, admin/usuario-form
<code>detalle-producto.js</code>	Galería, control de cantidad, carga de relacionados	detalle-producto
<code>admin.js</code>	Toggle sidebar admin, toolbar, listeners comunes	todas las vistas admin
<code>admin-producto.js</code>	Validaciones producto-form, modo crear/editar, previsualización de imagen	admin/producto-form
<code>admin-usuario.js</code>	Validaciones usuario-form, modo crear/editar	admin/usuario-form

6.1 Reglas de validación ya documentadas en HTML

Los formularios ya exponen los atributos necesarios para la validación JS:

- `required`, `aria-required="true"` en campos obligatorios.
- `maxlength`, `minlength`, `min`, `max`, `step` como límites físicos.
- `inputmode` correcto por tipo de dato.
- `autocomplete` semántico (`name`, `email`, `new-password`, `current-password`, `tel`, `bday`, `street-address`).

- `aria-describedby` apuntando a mensaje de ayuda + error.
- `id` de cada `<p class="campo__error">` consistente: `{campo}-error`.

Esto permite que la capa JS sea declarativa y reutilizable mediante un helper de validación común.

7. Estructura de directorios del proyecto

```
proyecto/
├── index.html
├── productos.html
├── detalle-producto.html
├── nosotros.html
├── blogs.html
├── detalle-blog-1.html
├── detalle-blog-2.html
├── contacto.html
├── registro.html
├── login.html
├── admin/
│   ├── home.html
│   ├── productos.html
│   ├── producto-form.html
│   ├── usuarios.html
│   └── usuario-form.html
├── assets/
│   ├── css/
│   │   ├── variables.css ← PENDIENTE
│   │   ├── styles.css ← PENDIENTE
│   │   └── admin.css ← PENDIENTE
│   ├── js/
│   │   ├── productos.js ← PENDIENTE
│   │   ├── carrito.js ← PENDIENTE
│   │   ├── main.js ← PENDIENTE
│   │   ├── contacto.js ← PENDIENTE
│   │   ├── registro.js ← PENDIENTE
│   │   ├── login.js ← PENDIENTE
│   │   ├── regiones-comunas.js ← PENDIENTE
│   │   ├── detalle-producto.js ← PENDIENTE
│   │   ├── admin.js ← PENDIENTE
│   │   ├── admin-producto.js ← PENDIENTE
│   │   └── admin-usuario.js ← PENDIENTE
│   └── img/
│       ├── favicon-perla.svg
│       ├── hero-perla-cosmica.webp
│       ├── producto-*.webp
│       ├── blog-*.webp
│       └── equipo-*.webp
```

8. Matriz de cumplimiento ERS — Capa HTML

Requerimiento	Estado	Observaciones
RF-01 Página Principal	✓ Completado	<code>index.html</code>
RF-02 Productos	✓ Completado	<code>productos.html</code>
RF-03 Detalle del Producto	✓ Completado	<code>detalle-producto.html</code>
RF-04 Carrito de Compras	⌚ Pendiente	No se entregó <code>carrito.html</code> en esta iteración
RF-05 Registro de Usuario	✓ Completado	<code>registro.html</code>
RF-06 Inicio de Sesión	✓ Completado	<code>login.html</code>
RF-07 Nosotros	✓ Completado	<code>nosotros.html</code>
RF-08 Blogs	✓ Completado	<code>blogs.html</code>
RF-09 Detalle de Blog	✓ Completado	<code>detalle-blog-1.html</code> , <code>detalle-blog-2.html</code>
RF-10 Contacto	✓ Completado	<code>contacto.html</code>
RF-11 Home Administrador	✓ Completado	<code>admin/home.html</code>
RF-12 Mantenedor de Productos	✓ Completado	<code>admin/productos.html</code> , <code>admin/producto-form.html</code>
RF-13 Mantenedor de Usuarios	✓ Completado	<code>admin/usuarios.html</code> , <code>admin/usuario-form.html</code>

Cobertura HTML: 12 de 13 requerimientos funcionales (92 %).

9. Riesgos identificados y mitigaciones

Riesgo	Impacto	Mitigación
Ausencia de CSS genera páginas en blanco en formularios admin	Alto	Priorizar creación de <code>variables.css</code> y <code>admin.css</code> como primer paso de la siguiente iteración
Ausencia de <code>carrito.html</code> deja RF-04 sin cubrir	Medio	Planificar en el siguiente sprint
Rutas de imágenes <code>assets/img/*.webp</code> podrían no existir aún	Medio	Implementar <code>onerror</code> en JS o usar placeholders mientras se generan los assets reales
Scripts JS con <code>defer</code> referenciados pero no creados generan errores 404 en consola	Bajo	Crear archivos vacíos con <code>console.info</code> para evitar 404 mientras se implementa la lógica

10. Recomendaciones finales

1. **Priorizar la creación de `variables.css`** como primer entregable de la siguiente iteración. Es la base de todo el sistema de diseño y desbloquea el resto de la capa CSS.
 2. **Implementar `admin.css` con la regla explícita de visibilidad** sobre `.formulario--admin` y `.formulario__grid` para resolver definitivamente el síntoma reportado.
 3. **Crear `carrito.html`** en la siguiente iteración para completar la cobertura del RF-04.
 4. **Validar contraste AA** con herramientas como WebAIM Contrast Checker antes de cerrar la fase CSS.
 5. **Mantener commits descriptivos** en GitHub, idealmente uno por archivo o por bloque funcional (ej: `feat(css): crear variables de identidad visual`, `feat(admin): añadir estilos de sidebar y topbar`).
 6. **Ejecutar la auditoría HTML5** con el validador oficial del W3C (`validator.w3.org`) antes de la entrega final.
-

11. Conclusión

La capa de estructura HTML del proyecto PokéTienda Pearl se encuentra **funcionalmente completa** para la Evaluación Parcial 1, con 15 archivos que cubren el 92 % de los requerimientos funcionales del ERS y cumplen con los estándares de semántica HTML5 y accesibilidad exigidos por la rúbrica.

El síntoma reportado por el cliente (páginas en blanco en `producto-form.html` y `usuario-form.html`) quedó **diagnosticado y atribuido a la ausencia de la capa CSS**, no a un error del HTML. Adicionalmente, se aplicaron correcciones de mejora semántica en las cinco vistas administrativas.

La siguiente iteración debe centrarse en la **capa de presentación (CSS)** siguiendo los lineamientos de la identidad visual v2.0 y, posteriormente, en la **capa de comportamiento (JavaScript)** conforme a las reglas de validación del ERS.

Elaborado por: Desarrollador Fullstack contratado

Revisado por: Equipo de desarrollo DSY1104

Estado del documento: Aprobado para archivo

Próxima revisión: Al cierre de la capa CSS